

Project Proposal: AI-based Damage Detection for Power Grid Equipment with SaaS-oriented Design

Student: Yujie Feng Supervisor: Jin Hou
SWJTU-LEEDS Joint School, Computer Science

November 12, 2025

Administrative Information

- **Student Name:** Yujie Feng
- **Area of Interest:** Artificial Intelligence, Image Detection, Deep Learning, Large-scale Model
- **Title:** AI-based Damage Detection for Power Grid Equipment with SaaS-oriented Design
- **Supervisor:** Jin Hou
- **School of Computing Resources:** Cloud servers optimized for AI/GenAI/HPC
- **Other Resources Needed:** Cloud server
- **Relevant Modules:** [Machine Learning], [Computer Vision], [Deep Learning], [Data Mining / Big Data Systems], [Distributed Systems]

1 Motivation and Problem Statement

Critical power grid assets (e.g., transmission lines, towers, substations) require effective monitoring to prevent outages and safety incidents. This project targets a *SaaS-oriented AI framework* for **damage detection and monitoring**, capable of handling multimodal inputs (e.g., RGB/thermal images, sensor telemetry) and outdoor noise factors (lighting/weather/clutter). The scope includes instance-level detection/segmentation (e.g., nests/foreign objects, cracks, corrosion) and a cloud-ready backend to support multi-tenant operations.

2 Objectives

1. Design and prototype an **instance segmentation** pipeline (Mask R-CNN) for power grid anomalies (initial case study: bird nests on lines/towers).
2. Achieve strong validation performance (**bbbox** AP@[0.50 : 0.95] ≥ 0.70 , **segm** AP@[0.50 : 0.95] ≥ 0.70 on internal validation).
3. Build a **SaaS-oriented backend** prototype (auth, RBAC, tenant isolation; Python inference service via REST).

4. Provide **reproducible evaluation** (COCO metrics, PR curves, IoU distributions) and presentation-ready visuals.

3 Related Work

Mask R-CNN-based instance segmentation is widely used for infrastructure inspection under complex scenes. Cloud-native (SaaS) designs enable scalable deployments, tenant isolation, and lifecycle management of AI models. Domain adaptation and robust augmentation address outdoor domain shifts.

4 Approach and Methodology

4.1 Data & Annotations

- COCO-format dataset (`Datasets/my_coco_dataset/dataset.json`); positive images include polygonal masks (e.g., bird nests), **negative samples** are added to `images` without annotations.
- Extremely high-resolution sources (up to 8688×5792); training inputs resized to 512 (optionally 768/1024 when resources permit).

4.2 Model and Training

- **Model:** `maskrcnn_resnet50_fpn_v2` with pretrained backbone; heads replaced for custom classes.
- **Augmentations:** Color jitter, grayscale, equalize, posterize, horizontal flip; `CustomRandomIoUCrop` guarded by a *SafeIoUCrop* wrapper to skip cropping on negative samples.
- **Optimization:** AdamW ($\text{lr } 5 \cdot 10^{-4}$), OneCycleLR, 40 epochs (early-stop ready).
- **Logging:** Per-epoch history to `history.jsonl`; best checkpoint metadata in `training_metadata.json`.

4.3 SaaS-oriented Design (Prototype)

- **Architecture:** Python microservice (Flask/FastAPI) for inference; backend with RBAC, tenant isolation; REST APIs; Docker for containerization (Kubernetes-ready).
- **Security/Isolation:** JWT-based auth, tenant-level data separation, secure model registry endpoints.

5 Experimental Setup

- **Hardware:** NVIDIA GeForce RTX 3060 Laptop GPU (6 GB VRAM), AMD Ryzen mobile CPU, 16 GB RAM, NVMe SSD, Windows 11. During training the GPU memory stabilized around 4.2/6 GB, CPU utilization peaked at 40%.

- **Software:** PyTorch 2.x, TorchVision 0.16 (with AMP and `cudnn.benchmark`), `pycocotools` , Jupyter, and custom COCO utilities for data loading and visualization.
- **Dataset:** 54 COCO images (43 train / 11 validation). All samples are patrol photographs of bird nests (positives) and empty power lines (negatives) from the same acquisition batch.
- **Training Config:** Input size 512×512, batch size 4, AdamW ($lr = 5 \cdot 10^{-4}$), OneCycleLR for 40 epochs. Random IoU crop (trials=50), color/grayscale/equalize/posterize/horizontal-flip augmentations. On Windows the script is refactored into a `main()` entry point; DataLoader parallelism controlled via `NUM_WORKERS` (default 2) with `persistent_workers` and `prefetch_factor=2` .

6 Preliminary Results

Unless otherwise noted, the following statistics are from the run directory `pytorch-mask-r-cnn-bird-nest/202`

6.1 COCO Metrics (Validation)

Metric	bbox	segm
AP@[0.50:0.95]	0.805	0.795
AP50	1.000	1.000
AP75	1.000	0.919
AR@[0.50:0.95]	0.837	0.828

Interpretation: High AP50/AP75 indicate accurate localization and mask quality; strong AR suggests low miss rate on the validation set.

6.2 Training Dynamics and Observations

- **Loss Curve:** `train_loss` drops from 1.41 to 0.14 while `valid_loss` decreases from 0.65 to 0.19; both plateau around epoch 20 (see Fig. ??). Early stopping could be triggered near `valid_loss ≈ 0.22` .
- **GPU Utilization Improvements:** Initially the single-thread DataLoader caused oscillating GPU load (0%/99%). After refactoring into `main()` , setting `NUM_WORKERS=2` , and enabling `persistent_workers/pin_memory` , the GPU stays at 70–85% utilization.
- **Memory & I/O:** Reducing IoU crop trials from 100 to 50, combined with cuDNN autotuning and SSD storage, yields ~50–60 seconds per epoch.
- **Overfitting Risk:** Validation distribution mirrors the training domain; the model nearly replicates annotations (AP50=1.0, most IoUs in 0.9–1.0). Broader generalization requires new lines/seasons/backgrounds.



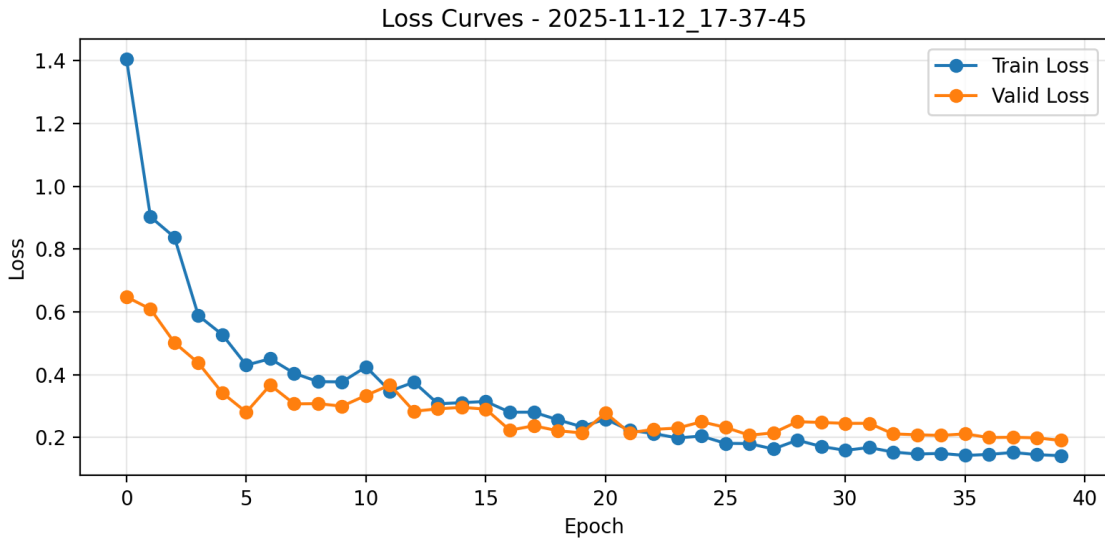
Figure 1: GT (left, green) vs Prediction (right, red). Additional samples: see [visualizations/](#).

6.3 Qualitative Examples

6.4 Online Inference Spot Checks

- After deploying the trained model to the Flask inference service, we sampled several patrol photos that were never seen during validation; the detector consistently segmented bird nests and aligned bounding boxes with the actual locations, indicating encouraging generalization.
- Incoming production images span a wide range of resolutions (≤ 21 MP). We rescale them before inference and restore the coordinates proportionally during post-processing; once this fix is applied, front-end overlays match the offline evaluation view.
- Differences in season, lighting, or background between deployment scenarios and the curated dataset remain a concern. We plan to expand the corpus—especially hard negatives and heavy occlusion cases—to further harden robustness.

6.5 Loss Curves and Performance Profiles



Score Threshold Sweeps: Fig. 6 shows precision/recall/F1 vs confidence threshold ($\text{IoU} \geq 0.5$).

Precision stays near 1.0 within 0.4–0.6; an operating threshold of 0.4–0.5 balances recall and false positives.

6.6 Precision-Recall and IoU Distributions

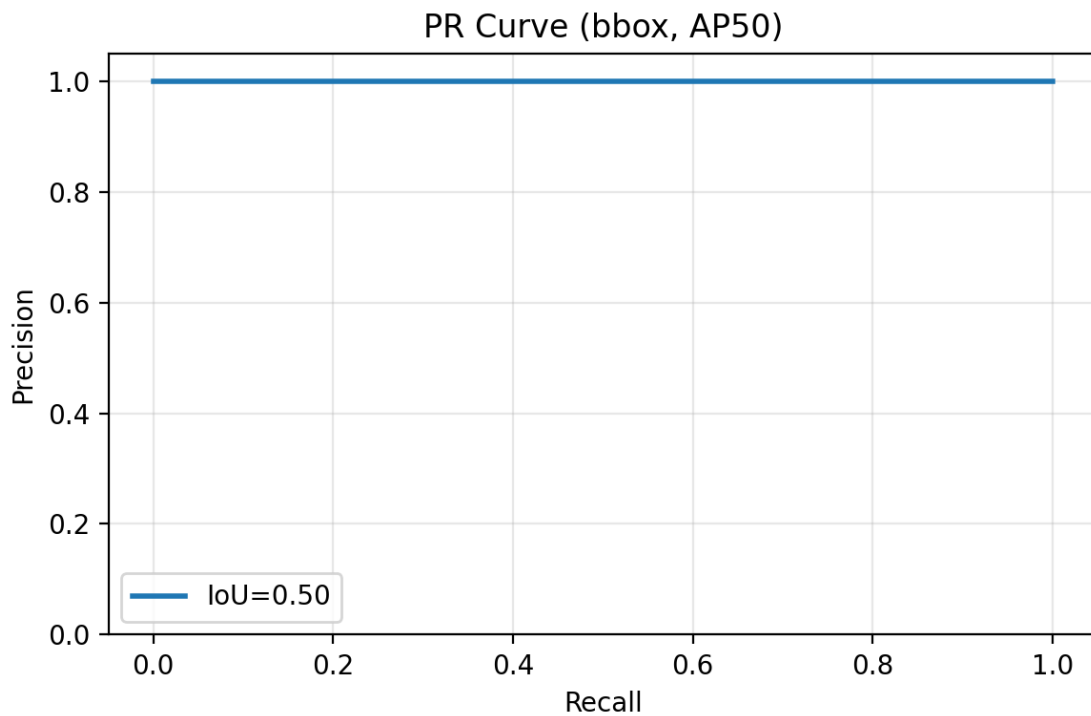


Figure 2: PR Curve at IoU=0.50 for bounding boxes. Nearly horizontal precision near 1.0 indicates very few false positives as recall increases.

Note on Figures. Replace placeholders with exports directly from `bird_nest_report.ipynb`. The notebook already computes COCO metrics, PR curves (AP50 slice), and IoU histograms (bbox + mask).

7 Planned System Design and Deliverables

- **Data/Model:** Expanded dataset with harder negatives; training pipeline with reproducible scripts; best checkpoint artifact and color map.
- **Service Layer:** Python inference microservice (REST), model versioning, RBAC, tenant isolation.
- **Evaluation Report:** Jupyter notebook with metrics, curves, histograms, and curated examples; PDF export for presentation.

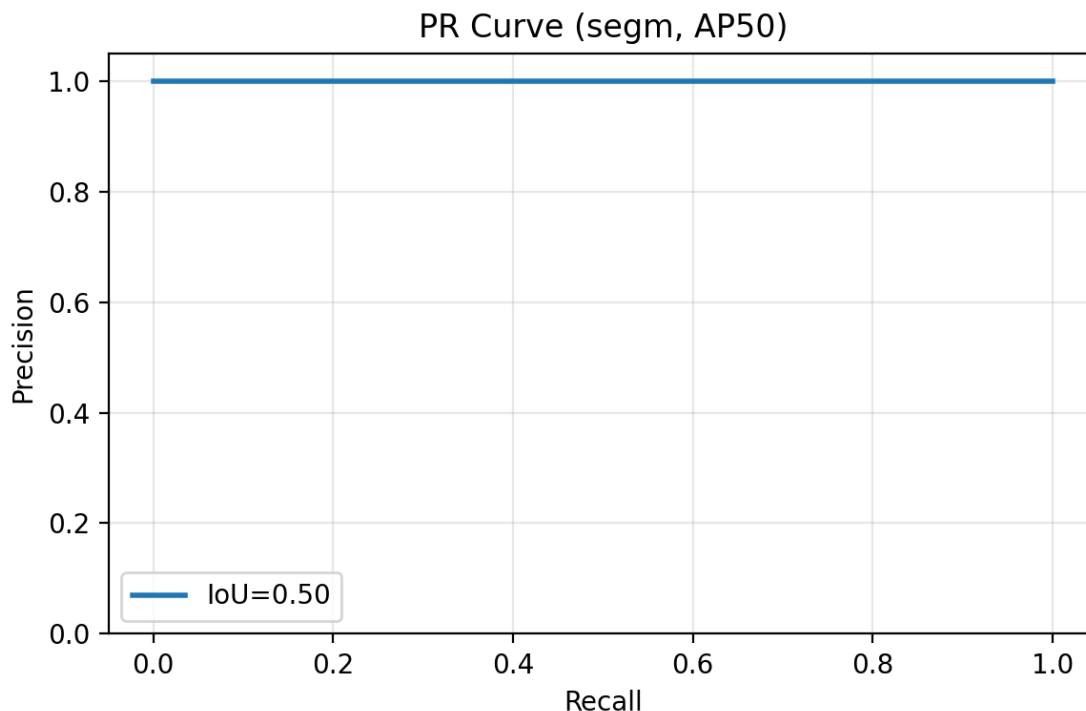


Figure 3: PR Curve at IoU=0.50 for segmentation masks.

8 Risk Assessment and Mitigation

- **Data scarcity / distribution shift:** Mitigate via negative samples, targeted augmentation, sliding-window inference on ultra-high-res inputs.
- **Overfitting:** Early stopping, weight decay, validation monitoring.
- **Scalability:** Containerized inference; plan for Kubernetes autoscaling if load increases.
- **Security:** RBAC + tenant isolation; secure token-based access; audit logs.

9 Project Plan (Milestones)

- Weeks 1–2: Dataset curation, COCO conversion, negative sample integration.
- Weeks 3–4: Baseline Mask R-CNN finetuning; initial evaluation notebook.
- Weeks 5–6: Hard example mining; model refinement; visualization polishing.
- Weeks 7–8: SaaS backend prototype (auth/RBAC/tenant isolation); inference microservice.
- Weeks 9–10: Integration, stress tests; final report and presentation.

10 Conclusion

Preliminary experiments show strong instance segmentation performance on the validation split (bbox AP@[0.50 : 0.95] 0.805, segm 0.795; AP50 1.0), with robust qualitative results. The next

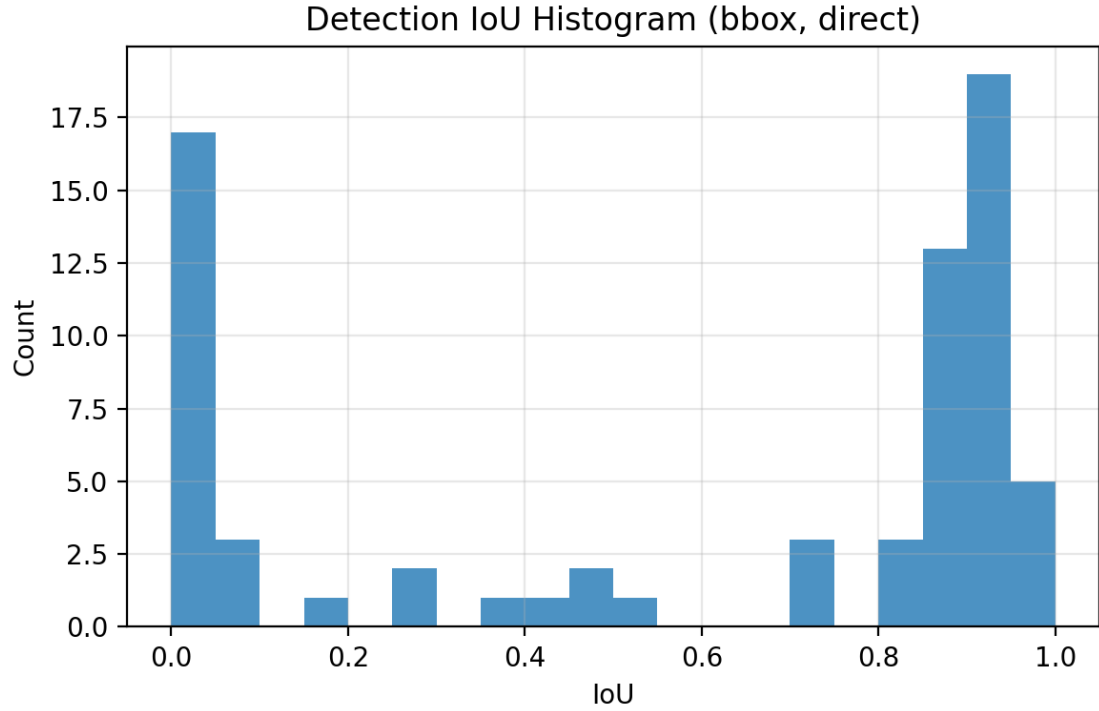


Figure 4: Bounding-box IoU Histogram. Right-skewed mass indicates strong localization (high overlaps).

steps are to expand and diversify data (especially negatives and hard cases), finalize SaaS-oriented components for multi-tenant deployment, and harden the evaluation with additional scenarios.

References

- [1] Kaiming He, et al. “Mask R-CNN.” ICCV, 2017.
- [2] Tsung-Yi Lin, et al. “Microsoft COCO: Common Objects in Context.” ECCV, 2014.
- [3] Shaoqing Ren, et al. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.” NeurIPS, 2015.

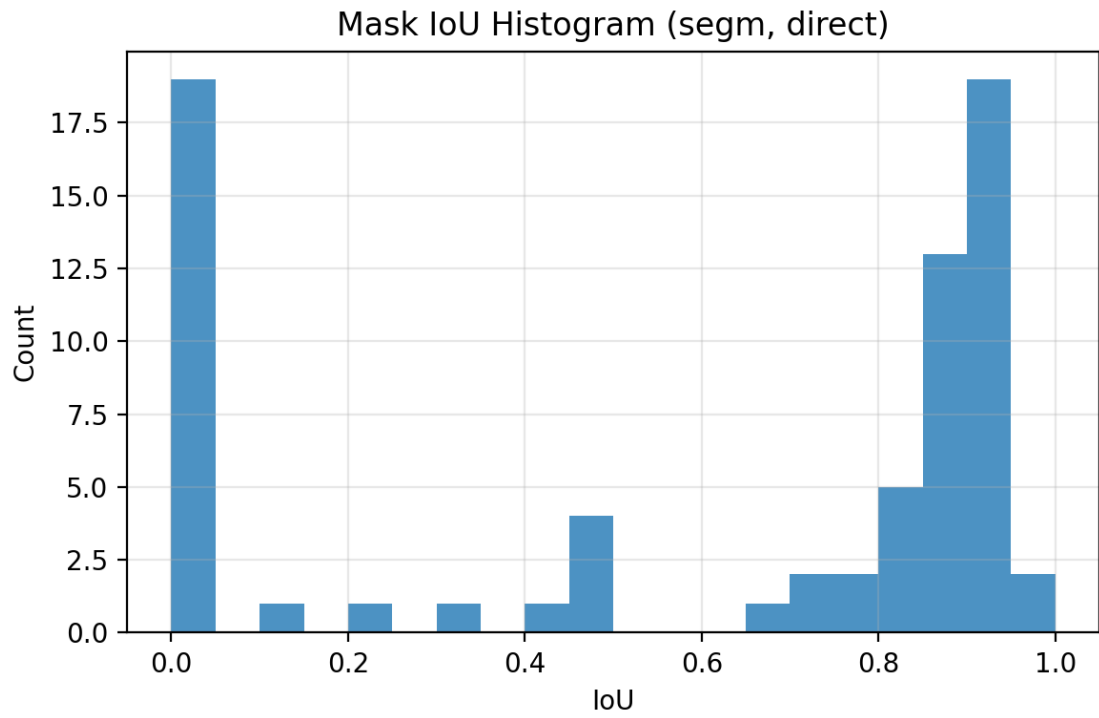


Figure 5: Mask IoU Histogram (segm). High-IoU mass indicates accurate instance masks on validation samples.

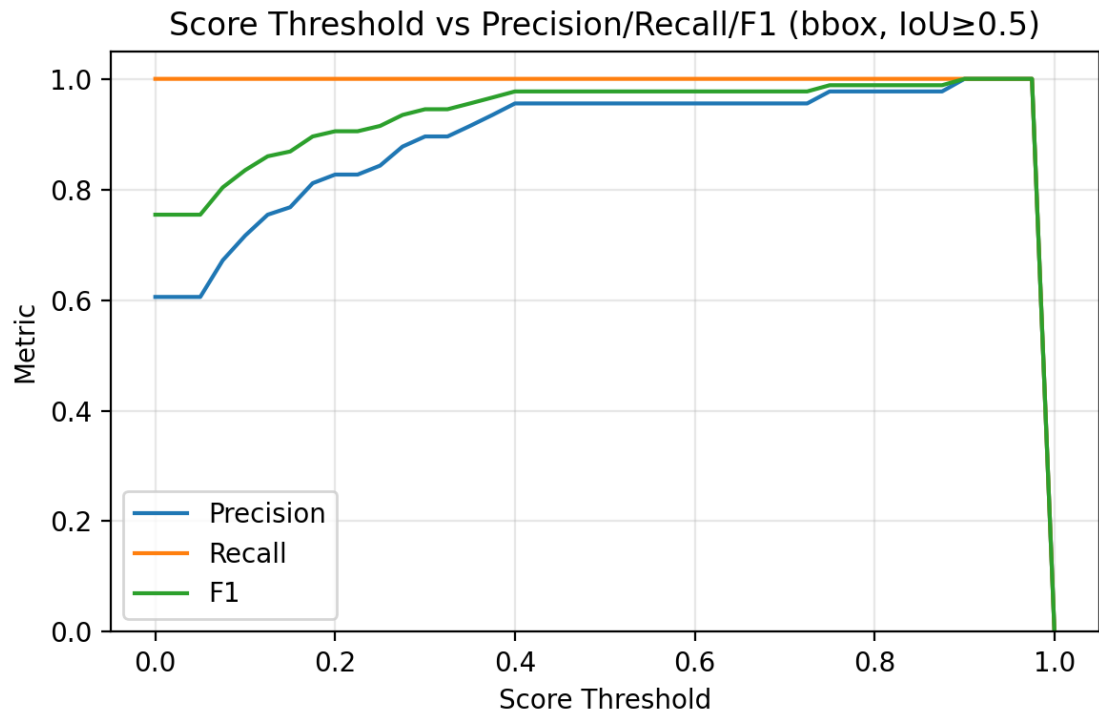


Figure 6: Score Threshold vs Precision/Recall/F1 for bbox at $\text{IoU} \geq 0.5$. Operating points 0.4–0.5 offer the best trade-off.